# Bitácora de Refactorización — Lambda Functions

## CONNECTA · Avance 7 · Guía Práctica N°. 7

**Proyecto:** CONNECTA — Sistema Inteligente de Gestión Veterinaria
**Autores:** Erick Ocampo, Daniel Arteaga, Andrés Rodríguez
**Sesión:** 10/04/2026
**Paradigma:** Programación Funcional — Python 3.11


**Bitácora de Refactorización — Lambda Functions** **1**
CONNECTA · Avance 7 · Guía Práctica N°. 7 1
1. Qué es una Lambda Function 1
2. Criterios de Selección — cuándo usar lambda 2
3. Bitácora de Decisiones 3
Sesión 10/04/2026 — Análisis inicial 3
4. Tabla Comparativa Antes vs. Después 5
4.1 Filtros — filter() + lambda 5
4.2 Transformaciones — map() + lambda 6
4.3 Ordenamientos — sorted() + lambda 8
4.4 Resumen cuantitativo 9
5. Análisis por Categoría 9
5.1 filter() — por qué reemplaza el for + if 9
5.2 map() — por qué reemplaza el for + append 10
5.3 sorted() + key — por qué reemplaza la función auxiliar 10
6. Conclusiones Técnicas 10
Lo que cambió 10
Limitaciones de lambda reconocidas 11
Principio central aplicado 11


## 1. Qué es una Lambda Function

Una **lambda function** en Python es una función anónima de una sola expresión, definida sin la palabra clave `def`. Su sintaxis es:

lambda parámetros: expresion

A diferencia de `def`, una lambda:

- No tiene nombre propio (aunque puede asignarse a una variable)
- No puede contener sentencias (`if`, `for`, `return` explícito)
- Retorna implícitamente el resultado de la expresión
- Está diseñada para usarse como argumento de funciones de orden superior (`filter`, `map`, `sorted`, `reduce`)

**Ejemplo conceptual:**

```
# Con def

def al_cuadrado(x):

    return x ** 2

# Con lambda (equivalente)

al_cuadrado = lambda x: x ** 2
```

## 2. Criterios de Selección — cuándo usar lambda

Durante el análisis del código de CONNECTA (Avances 2–6) se aplicaron los siguientes criterios para decidir qué lógica era candidata a lambda:

| Criterio | ¿Lambda? | Justificación |
|---|---|---|
| Lógica de 1 expresión simple | ✅ Sí | Lambda es más concisa, no necesita `def` |
| Se usa como argumento de `filter`/`map`/`sorted` | ✅ Sí | Caso de uso natural de lambda |
| Lógica reutilizada en 5+ lugares | ⚠️ Evaluar | Preferir `def` con nombre descriptivo |
| Contiene más de 1 condición compuesta | ❌ No | Usar `def` para preservar legibilidad |
| Requiere manejo de errores (`try/except`) | ❌ No | Lambda no soporta bloques de código |
| Lógica de negocio crítica | ❌ No | `def` con docstring es más mantenible |

---

## 3. Bitácora de Decisiones

### Sesión 10/04/2026 — Análisis inicial

**09:00 — Revisión del código existente (Avances 2–6)**

Se revisaron los archivos:

- `actividades/main.py` — 659 líneas, lógica de colecciones dispersa en `main()`
- `actividades/avance3/decision_engine.py` — 5 reglas de negocio con estructuras `if/elif`

**Observación clave:** Los loops `for` del `main()` en Avance 2 aplican la misma operación a toda una colección — patrón exacto que `map()` y `filter()` resuelven de forma más expresiva.

**Código original identificado como candidato:**

```
# actividades/main.py — líneas 583-585 (Avance 2, sin refactorizar)

print(f"\n  PACIENTES ({len(patients)}):")

for p in patients:

    print(f"    [{p['id']}] {p['name']} - {p['breed']} | ...")
```

Este patrón se repite 5 veces (pacientes, servicios, citas, veterinarios, FAQ). Cada iteración aplica la misma transformación: `dict → string`. → **Candidato a `map()`**.

**09:30 — Identificación de filtros implícitos**

En `avance3/decision_engine.py` se detectó lógica de filtrado de citas por estado dentro de funciones más grandes. Extraer esos predicados como lambdas permite usarlos por composición.

**Decisión:** Crear lambdas nombradas (`filtrar_confirmadas`, `filtrar_saludables`) en lugar de repetir el predicado `p['health_status'] == 'Saludable'` en múltiples `for`.

**10:00 — Diseño del módulo `functions.py`**

Se decidió separar claramente dos grupos:

ACTIVIDAD 1: funciones def → lógica de negocio, validaciones, cálculos

ACTIVIDAD 2: lambda → predicados de filtro, transformaciones, criterios de orden

**Regla de diseño adoptada:** Si la lógica cabe en una línea y opera sobre una colección completa, va como lambda. Si necesita más de una línea o maneja errores, va como `def`.

**10:45 — Decisiones sobre `sorted()` + lambda**

El patrón `sorted(lista, key=lambda x: x['campo'])` fue el más discutido. Se evaluaron dos alternativas:

# Alternativa A — operator.itemgetter (más rápida en CPython)

from operator import itemgetter

sorted(servicios, key=itemgetter('price'))

# Alternativa B — lambda (más legible, sin import extra)

sorted(servicios, key=lambda s: s['price'])

**Decisión:** Se adoptó la alternativa B (lambda) por ser el objeto de estudio de la guía y porque la diferencia de rendimiento es despreciable en colecciones pequeñas (<100 elementos).

**11:15 — Refactoring de `main.py` → `run()`**

Se comprobó que con las lambdas y funciones `def` ya definidas, el nuevo `main.py` podía escribirse como una secuencia pura de llamadas a funciones. La función `run()` quedó sin ninguna lógica propia: solo coordina.

**Resultado:** `run()` = 8 líneas de código, todas llamadas a funciones. Cero lógica dispersa.

# 4. Tabla Comparativa Antes vs. Después

## 4.1 Filtros — `filter()` + lambda

| # | Operación | ANTES (Avance 2) | DESPUÉS (Avance 7) |
|---|---|---|---|
| F1 | Filtrar pacientes caninos | `for p in patients:` + `if p['species'] == 'Canino':` | `filtrar_caninos(pacientes)` |
| F2 | Filtrar pacientes felinos | `for p in patients:` + `if p['species'] == 'Felino':` | `filtrar_felinos(pacientes)` |
| F3 | Filtrar pacientes saludables | `for p in patients:` + `if` | `filtrar_saludables(pacientes)` |

| # | Operación | ANTES (Avance 2) | DESPUÉS (Avance 7) |
|---|---|---|---|
| | | `p['health_status'] == 'Saludable':` | |
| F4 | Filtrar citas confirmadas | `for a in appointments:` + `if a['status'] == 'Confirmada':` | `filtrar_citas_confirmadas(citas)` |
| F5 | Filtrar citas pendientes | `for a in appointments:` + `if a['status'] != 'Confirmada':` | `filtrar_citas_pendientes(citas)` |

**Código expandido — F3 (filtrar saludables):**

```
# ── ANTES ──────────────────────────────────────────────────────────

# actividades/main.py (estilo Avance 2)

saludables = []

for p in patients:

    if p['health_status'] == 'Saludable':

        saludables.append(p)

# 4 líneas, variable auxiliar, mutación de lista externa

# ── DESPUÉS ────────────────────────────────────────────────────────

# actividades/avance7/functions.py

filtrar_saludables = lambda pacientes: list(

    filter(lambda p: p["health_status"] == "Saludable", pacientes)

)
```

```
saludables = filtrar_saludables(pacientes)

# 1 expresión reutilizable, sin mutación, sin variable auxiliar
```

**Diferencia clave:** La versión lambda es una **expresión pura** — no muta estado externo, no requiere variable auxiliar, puede pasarse como argumento a otra función.

## 4.2 Transformaciones — `map()` + lambda

| # | Operación | ANTES (Avance 2) | DESPUÉS (Avance 7) |
|---|---|---|---|
| M1 | Extraer nombres de pacientes | `for p in patients: nombres.append(p['name'])` | `obtener_nombres_pacientes(pacientes)` |
| M2 | Calcular precios con IVA | `for p in precios: resultado.append(round(p * 1.19, 2))` | `calcular_precios_con_iva(precios)` |
| M3 | Extraer precios de servicios | `for s in services: precios.append(s['price'])` | `extraer_precios_servicios(servicios)` |

**Código expandido — M2 (calcular precios con IVA):**

```
# ── ANTES ──────────────────────────────────────────────────────────────

# Estilo imperativo (Avance 2 — sin refactorizar)

precios_con_iva = []

for precio in precios_base:

    precio_final = round(precio * 1.19, 2)

    precios_con_iva.append(precio_final)

# 4 líneas, variable temporal, acumulación manual
```

```
# ── DESPUÉS ──────────────────────────────────────────────────────────────

# actividades/avance7/functions.py

calcular_precios_con_iva = lambda precios: list(

    map(lambda precio: round(precio * 1.19, 2), precios)

)

precios_con_iva = calcular_precios_con_iva(precios_base)

# 1 expresión: la transformación está en la lambda, map la aplica a cada elemento
```

**Diferencia clave:** `map()` declara **qué** transformar, no **cómo** iterar. El loop es responsabilidad del intérprete — el programador solo describe la función de transformación.

## 4.3 Ordenamientos — `sorted()` + lambda

| # | Operación | ANTES (Avance 2) | DESPUÉS (Avance 7) |
|---|---|---|---|
| S1 | Servicios por precio ↑ | Sin implementar (lista en orden de inserción) | `ordenar_servicios_por_precio_asc(servicios)` |
| S2 | Servicios por precio ↓ | Sin implementar | `ordenar_servicios_por_precio_desc(servicios)` |
| S3 | Citas por hora | Sin implementar (orden de inserción) | `ordenar_citas_por_hora(citas)` |
| S4 | Pacientes por edad ↑ | Sin implementar | `ordenar_pacientes_por_edad(pacientes)` |
| S5 | Pacientes por peso ↓ | Sin implementar | `ordenar_pacientes_por_peso(pacientes)` |

**Código expandido — S2 (servicios por precio descendente):**

```
# ── ANTES ────────────────────────────────────────────────────────────────
```

```
# Estilo imperativo clásico con función auxiliar

def get_price(servicio):

    return servicio['price']

servicios_ordenados = sorted(services, key=get_price, reverse=True)

# Requiere definir una función auxiliar solo para acceder a un campo

# ── DESPUÉS
```

---

```
# actividades/avance7/functions.py

ordenar_servicios_por_precio_desc = lambda servicios: sorted(

    servicios, key=lambda s: s["price"], reverse=True

)

resultado = ordenar_servicios_por_precio_desc(servicios)

# El criterio de orden se expresa inline: no es necesaria una función auxiliar separada
```

**Diferencia clave:** El `key=lambda s: s["price"]` elimina la necesidad de una función auxiliar de un solo uso (`get_price`). La intención es legible en el sitio de la llamada.

## 4.4 Resumen cuantitativo

| Categoría | Líneas antes | Líneas después | Reducción | Funciones auxiliares eliminadas |
|---|---|---|---|---|
| Filtros (`filter`) | 20 líneas (5 × 4) | 5 expresiones lambda | −75% | 0 (no existían — loops inline) |
| Transformaciones (`map`) | 12 líneas (3 × 4) | 3 expresiones lambda | −75% | 0 (loops inline) |
| Ordenamientos (`sorted`) | 10 líneas (5 × 2) | 5 expresiones lambda | −50% | 5 (`get_price`, `get_age`, etc.) |
| **Total** | **~42 líneas** | **13 expresiones** | **−69%** | **5** |

---

# 5. Análisis por Categoría

## 5.1 `filter()` — por qué reemplaza el `for + if`

IMPERATIVO FUNCIONAL

---

```
resultado = []          ← init      filtrar_saludables = lambda ps: ...

for p in pacientes:     ← loop

    if condicion(p):    ← guard     # list(filter(predicado, ps))

        resultado.append(p) ← acum

                                    # predicado se reutiliza como valor
```

`filter()` abstrae el patrón "iterar + guardar si cumple condición". La lambda define el predicado como un valor de primera clase: puede pasarse, almacenarse y componerse.

## 5.2 `map()` — por qué reemplaza el `for + append`

IMPERATIVO FUNCIONAL

---

```
resultado = []          ← init      list(map(transformacion, coleccion))

for x in coleccion:     ← loop

    resultado.append(f(x)) ← acum
```

`map()` declara que **cada elemento** pasa por la misma transformación. Elimina el ruido de la acumulación manual y hace evidente que no hay efectos secundarios.

## 5.3 `sorted()` + `key` — por qué reemplaza la función auxiliar

`sorted()` con `key=lambda` permite expresar el criterio de orden en el mismo lugar donde se ordena, sin saltar a una función definida más arriba. En colecciones de diccionarios, es el patrón más idiomático de Python.

```
# El lector entiende el criterio sin buscar la definición de get_price

sorted(servicios, key=lambda s: s["price"], reverse=True)
```

# 6. Conclusiones Técnicas

## Lo que cambió

| Aspecto | Avance 2 (antes) | Avance 7 (después) |
|---|---|---|
| Paradigma de iteración | Imperativo (`for` + `append`) | Funcional (`filter`/`map`/`sorted`) |
| Variables auxiliares de acumulación | 1 por operación | 0 |
| Funciones auxiliares de un solo uso | 5 (para `sorted`) | 0 |
| Reutilización del predicado/clave | No — inline | Sí — lambda nombrada |
| Líneas de código para 13 operaciones | ~42 | 13 expresiones |
| Mutación de estado externo | Sí (`append` sobre lista propia) | No — retorno de nueva lista |

## Limitaciones de lambda reconocidas

| Limitación | Solución aplicada |
|---|---|
| No soporta `try/except` | Validaciones complejas se mantienen como `def` (ej: `validar_entrada_paciente`) |
| No soporta múltiples sentencias | Lógica de más de 1 expresión va en `def` (ej: `calcular_resumen_financiero`) |
| Difícil de depurar (sin nombre en traceback) | Se asignan a variables con nombre descriptivo (`filtrar_saludables`, no lambda anónima) |
| PEP 8 desaconseja asignar lambda a variable directamente | Decisión académica consciente: el objetivo de la guía es demostrar el patrón |

## Principio central aplicado

> **Las lambda functions no reemplazan `def` — complementan `filter`, `map` y `sorted` cuando el predicado o la clave es de un solo uso o lo suficientemente simple para expresarse inline.**

En CONNECTA, ambos coexisten: `def` para lógica de negocio con docstring, `lambda` para predicados de colección expresados en el punto de uso.